Infrastructure as Code - Provisioning Part 2

WELCOME TO CLASS 4!

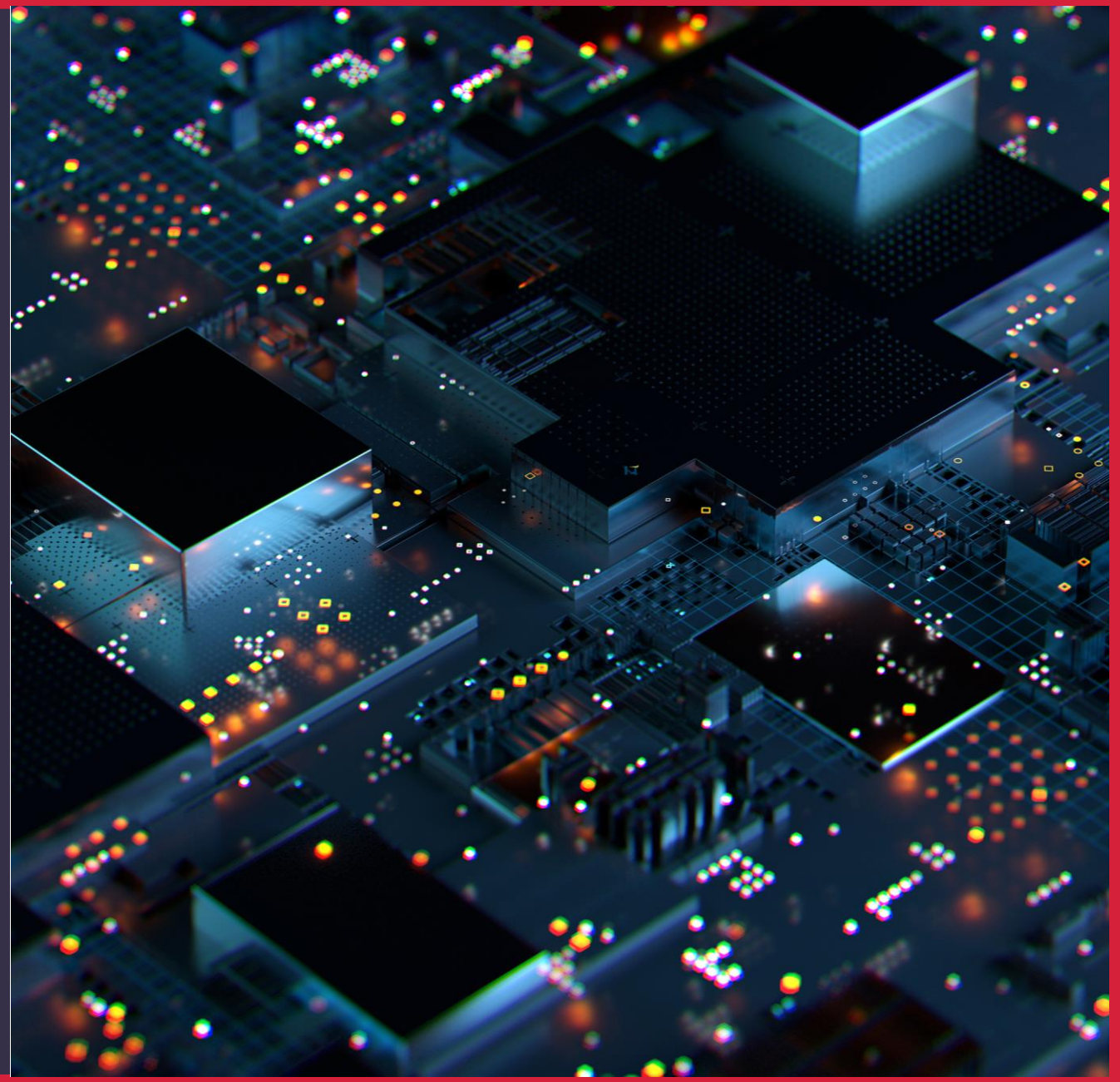


Photo by David Becker on Unsplash

Agenda

- Instructor led overview of Infrastructure as Code theory to be covered in this class
- Review Terraform state options
- Terraform modules
- **Break**
- Drift detection
- Testing your Terraform
- Terraform - DEV vs PROD

Goals of the day

- Deeper understanding of modules
- Terraform to create multiple environments
- Linting, testing and branching for a successful IaC practice

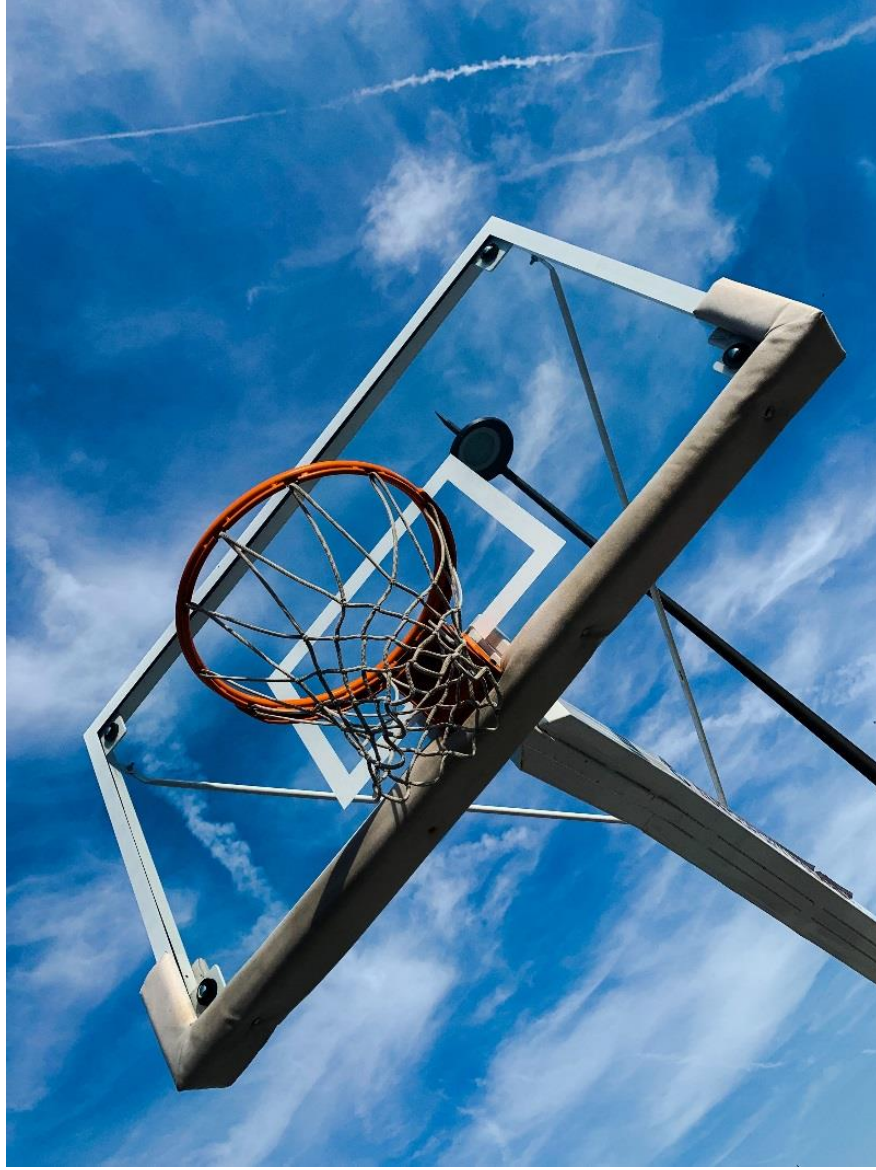
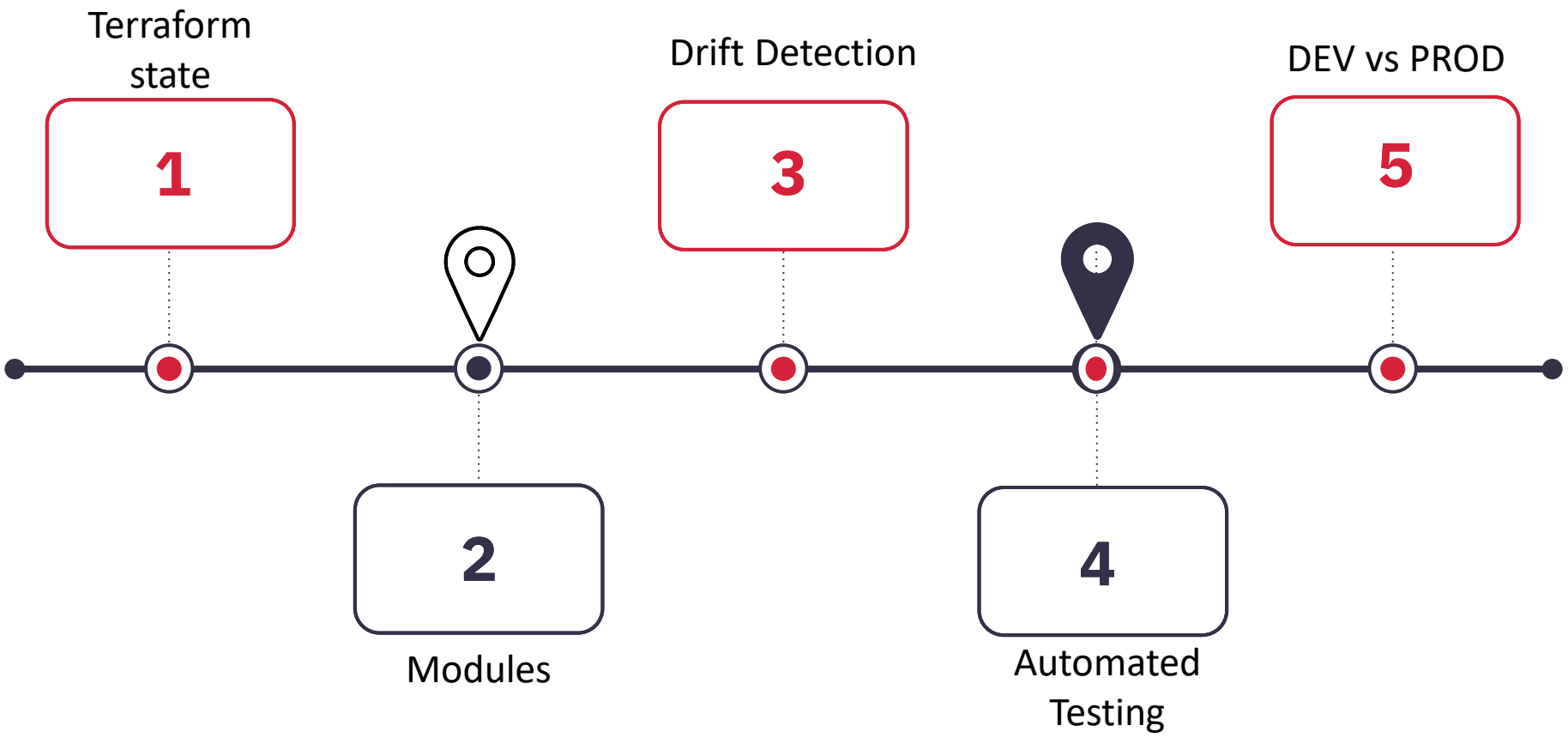


Photo by David Becker on Unsplash

Review

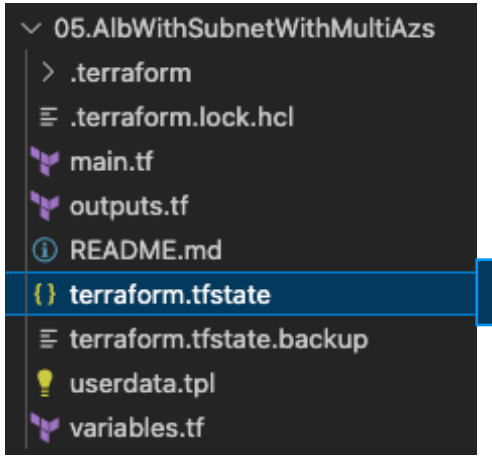
- How did the exercises from last class go for you?
- Did you notice any patterns as we progressed through these exercises?
- What would you differently with that code?
- What would you do more of?
- Brief Q & A

Journey Map


Terraform state
1
Drift Detection
3
DEV vs PROD
5
2
Modules
4
Automated Testing
Testing

Infrastructure as Code Theory – Terraform State

- In the exercises to date you may have noticed a file called **`terraform.tfstate`**
- It holds the current state of the infrastructure created by this terraform code, in detail
- Terraform references this file for plan, apply, state and destroy commands
- This works fine when you are running everything locally on your laptop but does not scale well when you need to collaborate with others



Infrastructure as Code Theory – Terraform State (cont)

- One option is to commit the state file to your SCM. However, this does not work well in practice
- The accepted way to manage state is to push this file to cloud storage, such as AWS S3
- Details of the terraform command shown here..
- During Terraform init, the connection to this storage location is made and the state file created
- In this way, when this code is subsequently run, by another developer or pipeline, the state is known/updated through this S3 state file

```
terraform {
  backend "s3" {
    bucket = "mybucket"
    key    = "path/to/my/key"
    region = "us-east-1"
  }
}
```

Terraform Modules

- As examples become more complex for the last class, you will have noticed that one main.tf file which contains all your code becomes unwieldy and complex.
- Using modules makes the code more readable and enables reuse of code as common resources required across multiple implementations.
- This approach allows an engineering team to write modules, that are signed off for use by operations or development team, which abstracts the complexity for these end users.
- With modules, there is a greater focus on up front design and standards, which helps with maintenance and adapting to change.
- Hashicorp also provides a Terraform Registry of common resources that can be used as part of your code.

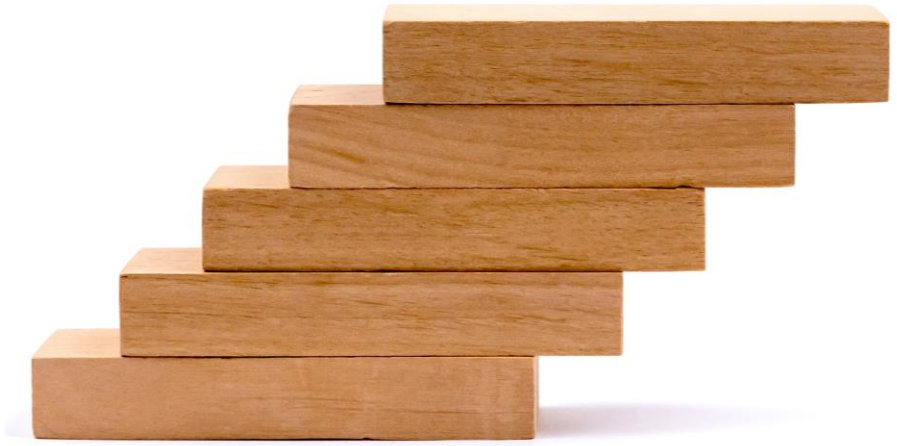


Photo by Volodymyr Hryshchenko on Unsplash

Terraform Modules Best Practice

In the same way as developer programming relies on packages, libraries and modules. Any real world Terraform implementation will use modules.

1. Begin coding with a modular design in mind – think in terms of inputs, outputs and resource dependencies. Try to group resources in a module, rather than have one resource per module.
2. Start with local modules and look for opportunities to turn these into remote modules (from git) to enable use by more than one implementation.
3. Check the Terraform registry for code to save you time writing your own.
4. Organize your module library so that it can be used effectively by others in your organization.

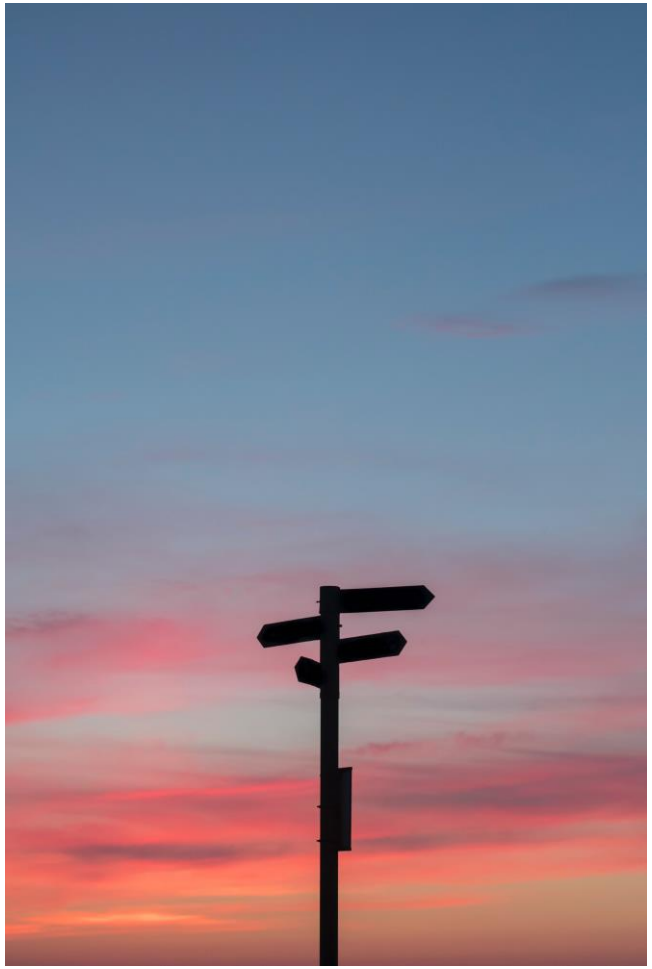
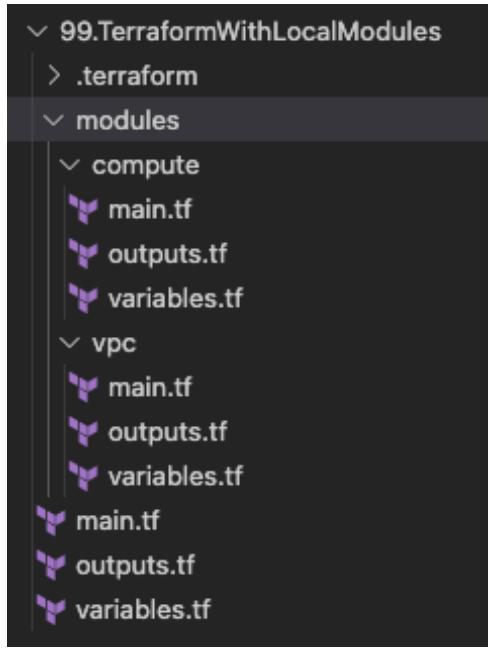


Photo by Javier Allegue Barros on Unsplash

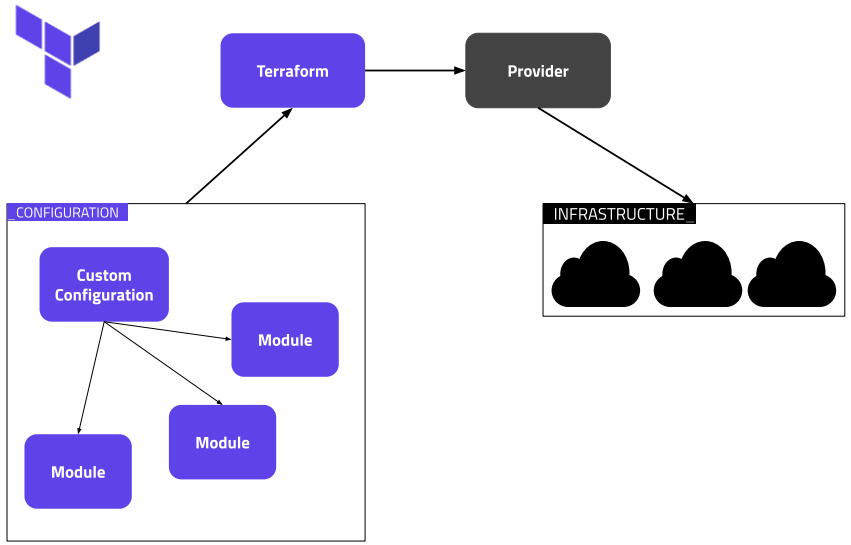
Infrastructure as Code Theory – Remote Modules

- So far, we have created local modules to refactor complex main.tf files
- To recap the benefits of modules that we have seen are:
  - Easier to navigate, understand and maintain
  - Creates an abstraction of a complex task that can be reused by many
  - Increases speed and stability of delivery based on proven, tested and documented code
- Using these local modules, we can progress to the next stage of Terraform evolution where these modules are stored separately in git and are termed remote



Infrastructure as Code Theory – Remote Modules (con't)

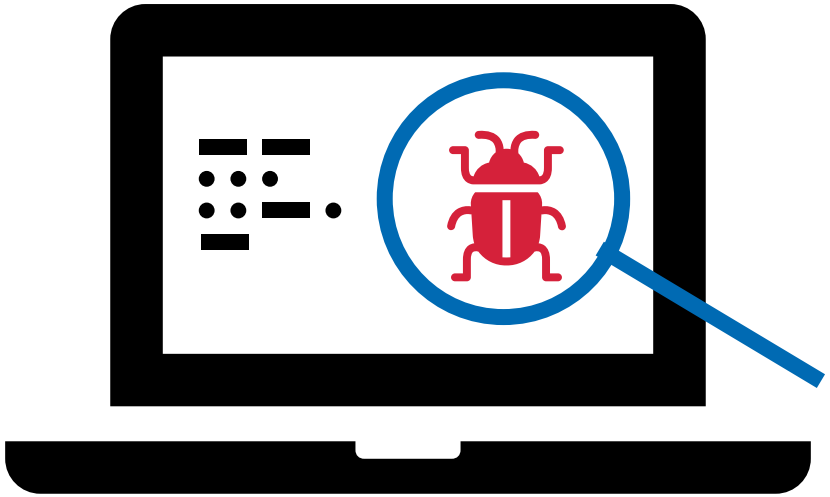
- In the coding world, the idea of breaking elements of a codebase into different packages or libraries is well understood
- The same can apply to Infrastructure as Code, we can break modules out into self contained blocks of code that can be imported by more than one project
- These modules have clearly defined inputs & outputs, which improves ease of reuse
- Applying version control best practice through a tool like git enables this software engineering approach to IaC to be developed further



https://www.hashicorp.com/blog/new-guides-terraform-modules

Infrastructure as Code Theory – Testing

- How can you ensure that changes you have made to your terraform code will not cause problems when run against PROD infrastructure?
- Like many DevOps process, IaC is improved through use of automated testing
- Unit, integration and end-to-end testing are staples of software engineering best practice
- How can we apply those practices here?



Infrastructure as Code Theory – Drift Detection

- In IaC terms, drift is the deviation between the defined state and the actual state of the infrastructure
- When created, the infrastructure exactly matches the terraform configuration
- If any change is made to the infrastructure through the cloud providers console or through any other method, then that change will be overwritten when the terraform is next applied
- Drift detection proactively monitors for any deviation and alerts on the matter immediately

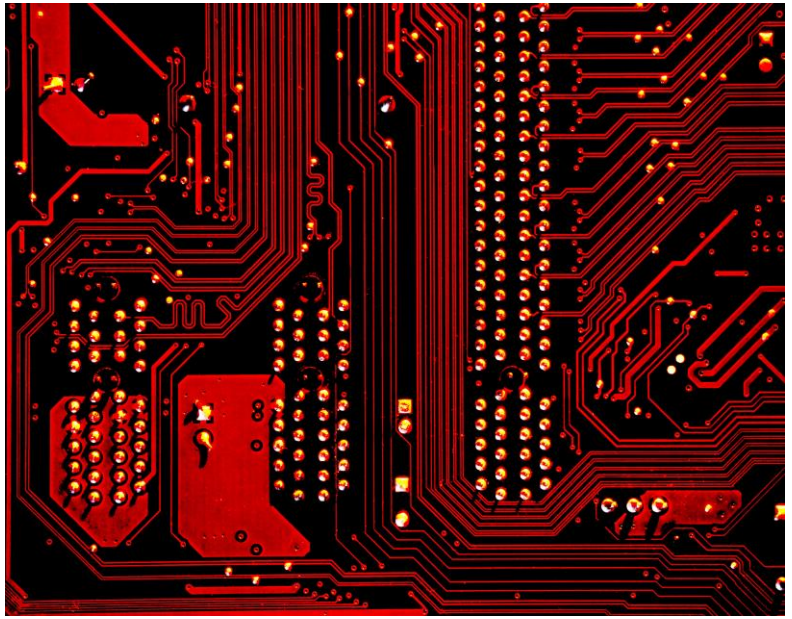
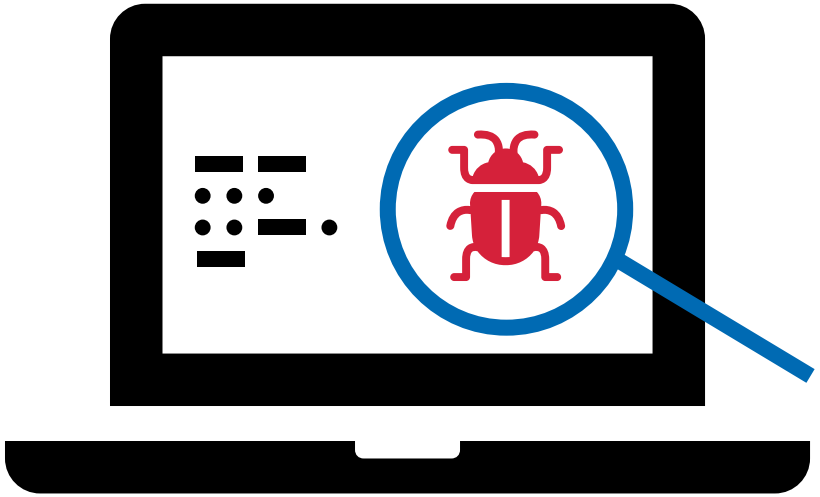


Photo by Michael Dziedzic on Unsplash

Infrastructure as Code Theory – Testing (con't)

Starting from the lowest level of granularity:

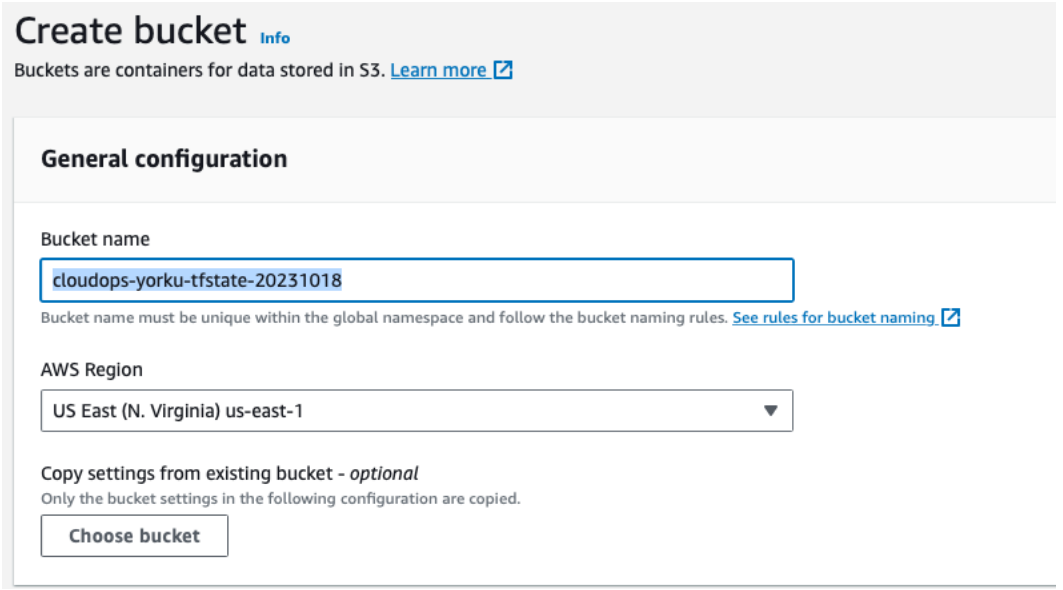
- The Commands **`terraform validate`** and **`terraform plan`** will highlight any syntax error in your code before you try to create any infrastructure
- Next you can run manual tests where you create infrastructure and systematically verify expected and actual results
- Alternatively, there are a range of tools such as Hashicorp Sentinel, Terratest



Exercise 1: Terraform State (1 of 4)

Code Overview

- As you have seen from previous examples, there is a lot of important information persisted in your terraform state files.
- In this exercise we will take the Simple Webserver code and update to use an S3 bucket to store state remotely.
- First log in to the AWS Console, navigate to S3 and click create new bucket
- The bucket name must be unique across all AWS, so pick a name that is specific to your example. Adding a date to the name can help find a unique, but meaningful name.
- Execute the code and verify that the remote entity is updated
- Clone the code to a different machine and show that it works as if it were run locally with the default state management e.g., change a security group config and apply

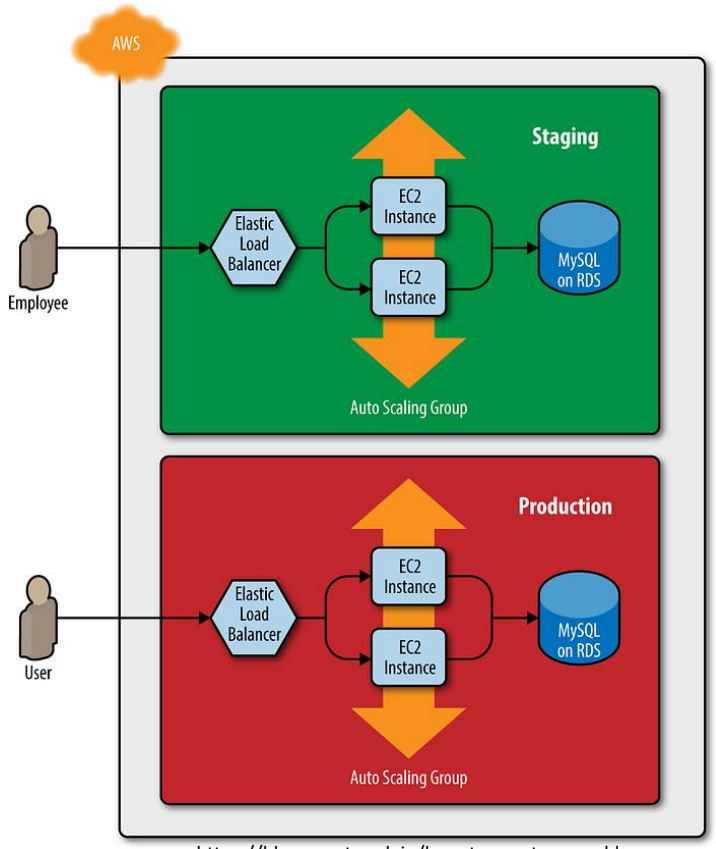


Infrastructure as Code Theory – DEV vs PROD

One of the benefits of IaC is consistency or parity across environments, which creates more stability and more predictable outcomes.

How is this achieved in practice through terraform?

- Well designed modules accept environment specific details as inputs and provide outputs that enable reuse
- Smaller VMs and lower scaling thresholds can be used in lower environments to save cost but maintain functional equivalence
- Versioning of modules enables new code to be developed and tested without impact to production code



https://blog.gruntwork.io/how-to-create-reusable-infrastructure-with-terraform-modules-25526d65f73d

Infrastructure as Code Theory – Best Practice

1. Follow a standard module structure.
2. Adopt a naming convention.
3. Use variables carefully.
4. Expose outputs.
5. Use data sources.
6. Limit the use of custom scripts.
7. Include helper scripts in a separate directory.
8. Put static files in a separate directory.
   Protect stateful resources.
9. Use built-in formatting.
10. Limit the complexity of expressions.
11. Use count for conditional values.
12. Use for_each for iterated resources.
13. Publish modules to a registry.

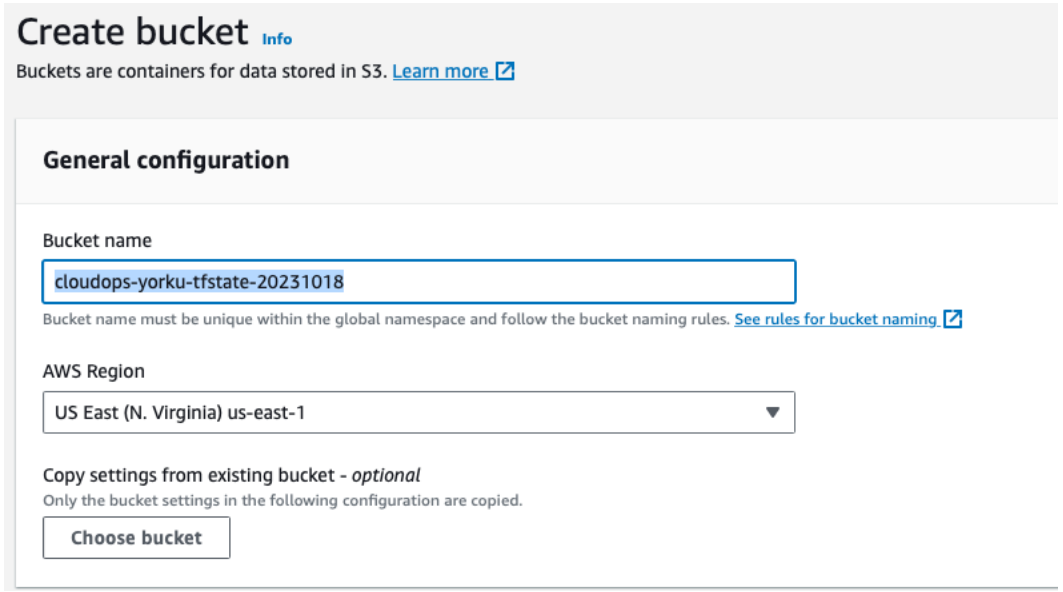
Photo by Sven Mieke on Unsplash

https://cloud.google.com/docs/terraform/best-practices-for-terraform

Exercise 1: Terraform State (2 of 4)

Code Overview

- As you have seen from previous examples, there is a lot of important information persisted in your terraform state files.
- In this exercise we will take the Simple Webserver code and update to use an S3 bucket to store state remotely.
- First log in to the AWS Console, navigate to S3 and click create new bucket
- The bucket name must be unique across all AWS, so pick a name that is specific to your example. Adding a date to the name can help find a unique, but meaningful name.
- Accept all defaults and click Create Bucket.



Exercise 1: Terraform State (3 of 4)

Code Overview

- The repo for this exercise takes the the SimpleWebServer code and adds a new file called backend.tf.
- Edit the file to update parameters for your implementation

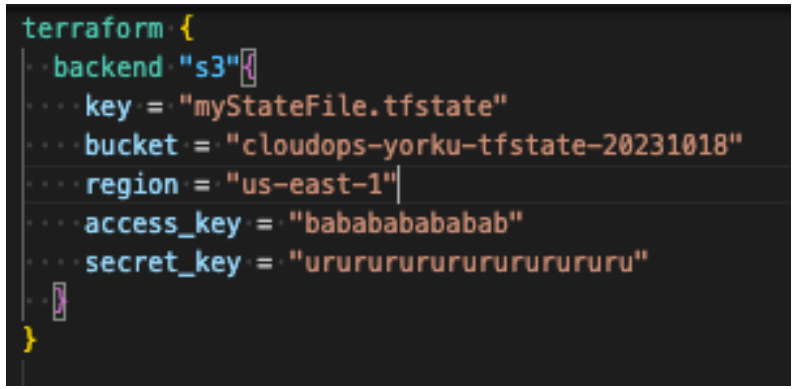
  `key` is your choice for name of the state file

  `bucket` is the one you just created

  `region` can be any region

  values for **`access_key`** and **`secret_key`** can be obtained by entering the following at the command line.

```
cat ~/.aws/credentials
```

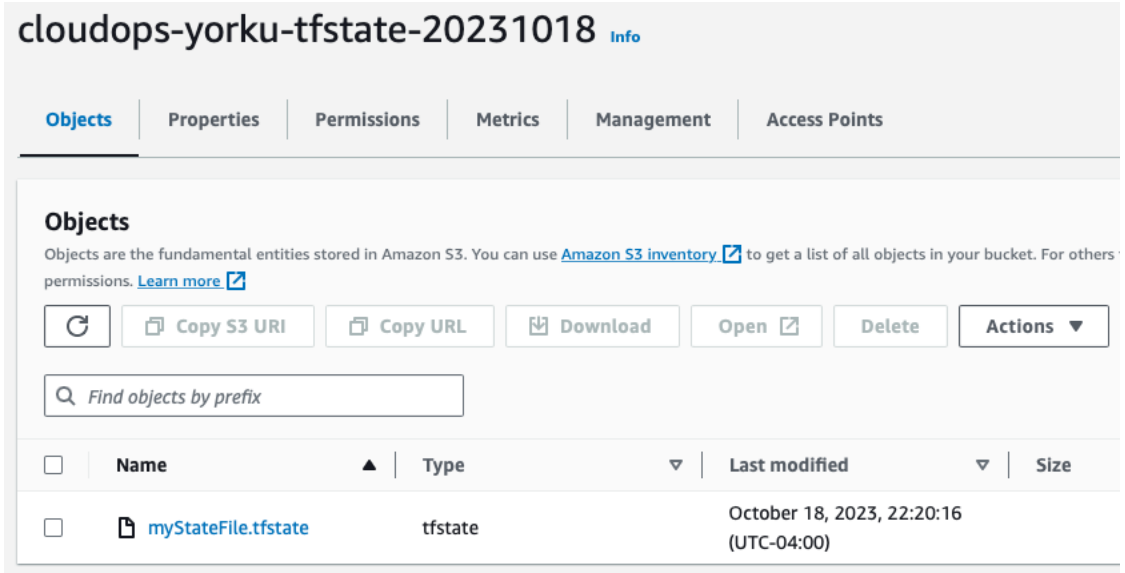


Exercise 1: Terraform State (4 of 4)

- Once finished editing, run a `terraform init` command and observe the results. Note the S3 backend was successfully configured.

```
shane@Shanes-MacBook-Pro Module4_Class4_Repo1_RemoteBackend % terraform init

Initializing the backend...

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.
```

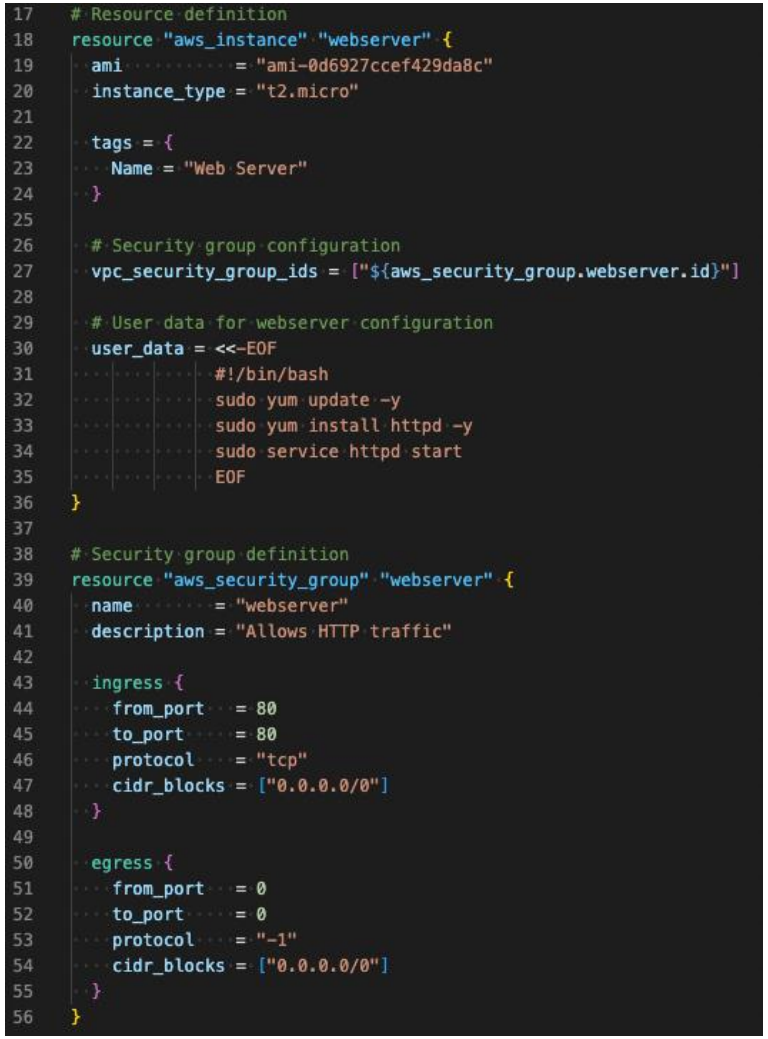
- Continue through to terraform apply.
- Check S3 bucket for new state file.
- This feature allows everyone on the team to reference one centralized state file.
- This allows teams to work at scale based on an automated solution, rather than use process steps to deal with this key issue.



Exercise 2: Modules (1 of 10)

Code Overview *

- The objective in this exercise is to create an EC2 with Apache httpd installed. We did this in Exercise 2 in Class 3.
- The screenshot shows the code we used in that exercise. It is quite straightforward.
- It uses the default VPC, but for this exercise we will create our own VPC and create the EC2 in that VPC.

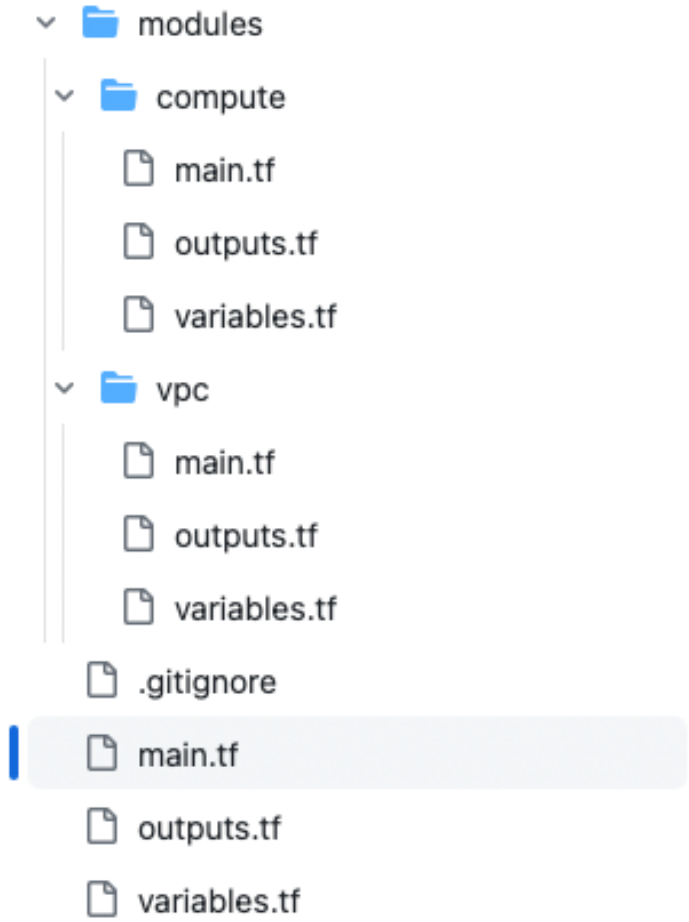


```
# Resource definition
resource "aws_instance" "webserver" {
  ami           = "ami-0d6927ccef429da8c"
  instance_type = "t2.micro"

  tags = {
    Name = "Web Server"
  }

  # Security group configuration
  vpc_security_group_ids = ["${aws_security_group.webserver.id}"]

  # User data for webserver configuration
  user_data = <<-EOF
              #!/bin/bash
              sudo yum update -y
              sudo yum install httpd -y
              sudo service httpd start
              EOF
}

# Security group definition
resource "aws_security_group" "webserver" {
  name        = "webserver"
  description = "Allows HTTP traffic"

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```


* https://github.com/SMannionYorkU/CloudOps_Module4_Class4_Repo2_WebServer_Modularized.git

Exercise 2: Modules (2 of 10)

- This screenshot shows the structure of the code.
- There are two modules, one to create the VPC and the other to create the EC2
- The main.tf at the top level of the code is executed when you run terraform apply and it has calls to main.tf files in the modules.
- This code will create an EC2 with Apache httpd running on it.
- One of the benefits of a modular design in that the VPC and Compute modules can also be used for other implementations.



Exercise 2: Modules (3 of 10)

- Reviewing the top level main.tf, you can see it is very easy to read, not excessively long nor cluttered like the more complex examples in the last class.
- It just calls on to the modules and they do the work to create the required infrastructure.
- First the VPC is created, then you can see in creating the compute resource that some of the vpc values are referenced. More on that later..

Class4 > Module4_Class4_Repo2_WebServerInModules > main.tf >

```
#-----main.tf-----
#==================
terraform {
  required_providers {
    aws = {
      version = "~> 3.44.0"
    }
  }

  required_version = ">= 0.15.5"
}

provider "aws" {
  region = var.region
}

#Deploy Networking Resources
#===========================
module "vpc" {
  source = "./modules/vpc"
}

#Deploy Compute Resources
#========================
module "compute" {
  source          = "./modules/compute"
  subnets         = module.vpc.public_subnets
  security_group  = module.vpc.public_sg
  subnet_ips      = module.vpc.subnet_ips
}
```

Exercise 2: Modules (4 of 10)

- Reviewing the VPC module main.tf, it is too long to fit in one screenshot, so some of the more common elements like security group have been collapsed.
- A number of network level elements are created here, not just the VPC.
- The approach is to push all of this complexity down into the module so that the consumer of it (main.tf at the top level) is not aware of the details. The user gets what they need without the details.

Exercise 2: Modules (5 of 10)

- Central to this module design is the inputs and outputs that are passed around, similar to request and response for an API call.
- For this module, there are no inputs passed from the calling code, but several outputs are returned.
- These outputs are retrieved by name from the module's main.tf and then returned to the calling main.tf.

Class4 > Module4_Class4_Repo2_WebServerInModules > modules > vpc > outputs.tf >

```
1   #-----vpc/outputs.tf-----
2   #========================
3
4   output "public_subnets" {
5     value = aws_subnet.tf_public_subnet.id
6   }
7
8   output "public_sg" {
9     value = aws_security_group.tf_public_sg.id
10  }
11
12  output "subnet_ips" {
13    value = aws_subnet.tf_public_subnet.cidr_block
14  }
15
```

Exercise 2: Modules (6 of 10)

- Have completed execution of the VPC code, we return to the top level main.tf and see that the outputs from the VPC module are referenced in the call to the compute module.
- The values are assigned during this call and so must be mapped within the compute module in order to be used in that code.

```
17  #Deploy Networking Resources
18  #============================
19  module "vpc" {
20    source = "./modules/vpc"
21  }
22
23  #Deploy Compute Resources
24  #============================
25  module "compute" {
26    source          = "./modules/compute"
27    subnets         = module.vpc.public_subnets
28    security_group  = module.vpc.public_sg
29    subnet_ips      = module.vpc.subnet_ips
30  }
```

Exercise 2: Modules (7 of 10)

- This assignment starts in the variables.tf of the called module.
- The variables without values here match the names used in calling the compute module from the top level main.tf
- Study this flow of variables up from one module, through the top level and then down into another module. Understanding this flow is critical to understanding the design of interdependent modules.

Class4 > Module4_Class4_Repo2_WebServerInModules > modules > compute > variables.tf

```
#-----compute/variables.tf-----
#=============================
variable "region" {
  type    = string
  default = "us-east-1"
}

variable "ssh_key_public" {
  type    = string
  #Replace this with the location of you public key .pub
  default = "~/.ssh/id_rsa.pub"
}

variable "ssh_key_private" {
  type    = string
  #Replace this with the location of you private key
  default = "~/.ssh/id_rsa"
}

variable "subnet_ips" {}

variable "security_group" {}

variable "subnets" {}

```

Exercise 2: Modules (8 of 10)

- The compute main.tf is not so complex.
- Where a variable starts with var. you can see that the value originates in the variables.tf, from outside the module, passed as an assignment.
- This code also includes configuration of SSH keys to allow connections to the EC2 we are creating.

Exercise 2: Modules (9 of 10)

- Values are returned from the compute module by routing through the outputs.tf file.
- From there, these values are subsequently referenced in the top level outputs.tf, to be included in output from the execution of this code.
- Don’t destroy this deployment yet, we will use it for the next exercise!

Class4 > Module4_Class4_Repo2_WebServerInModules > modules > compute > outputs.tf

```
#-----compute/outputs.tf-----
#============================
output "server_id" {
  value = aws_instance.webserver.id
}

output "server_ip" {
  value = aws_instance.webserver.public_ip
}
```

Class4 > Module4_Class4_Repo2_WebServerInModules > outputs.tf

```
#-----outputs.tf-----
#====================
output "Apache-Webserver-Public-URL" {
  value = "http://${module.compute.server_ip}"
}
```

```
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

Apache-Webserver-Public-URL = "http://54.89.230.232"
```

Exercise 2: Modules 10 of 10)

Important points to note from this exercise are as follows:

- The top level main.tf is very simple, because the complexity is hidden in the modules
- The variable.tf and outputs.tf are used by modules to facilitate data requested and data responses.
- Chaining through this mechanism enables sharing of data across modules.

Class4 > Module4_Class4_Repo2_WebServerInModules > modules > compute > outputs.tf

```
#-----compute/outputs.tf-----
#============================
output "server_id" {
  value = aws_instance.webserver.id
}

output "server_ip" {
  value = aws_instance.webserver.public_ip
}
```

Class4 > Module4_Class4_Repo2_WebServerInModules > outputs.tf

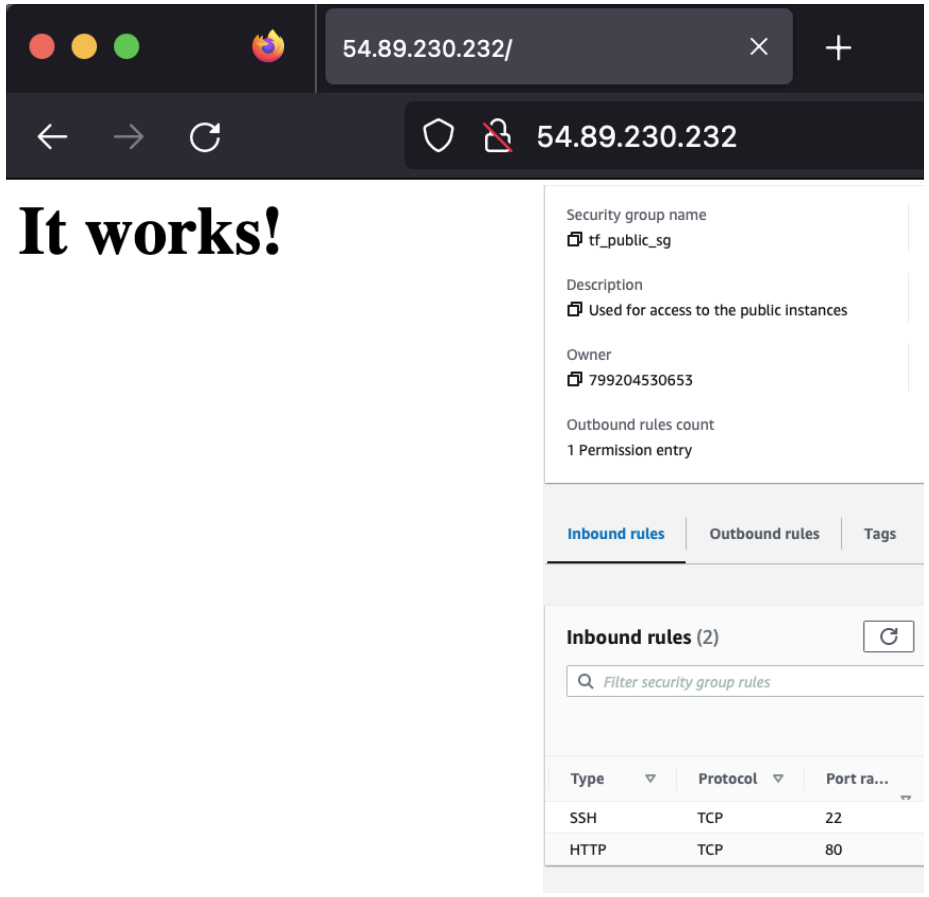
```
#-----outputs.tf-----
#====================
output "Apache-Webserver-Public-URL" {
  value = "http://${module.compute.server_ip}"
}
```

```
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

Apache-Webserver-Public-URL = "http://54.89.230.232"
```

Break (15 Minutes)

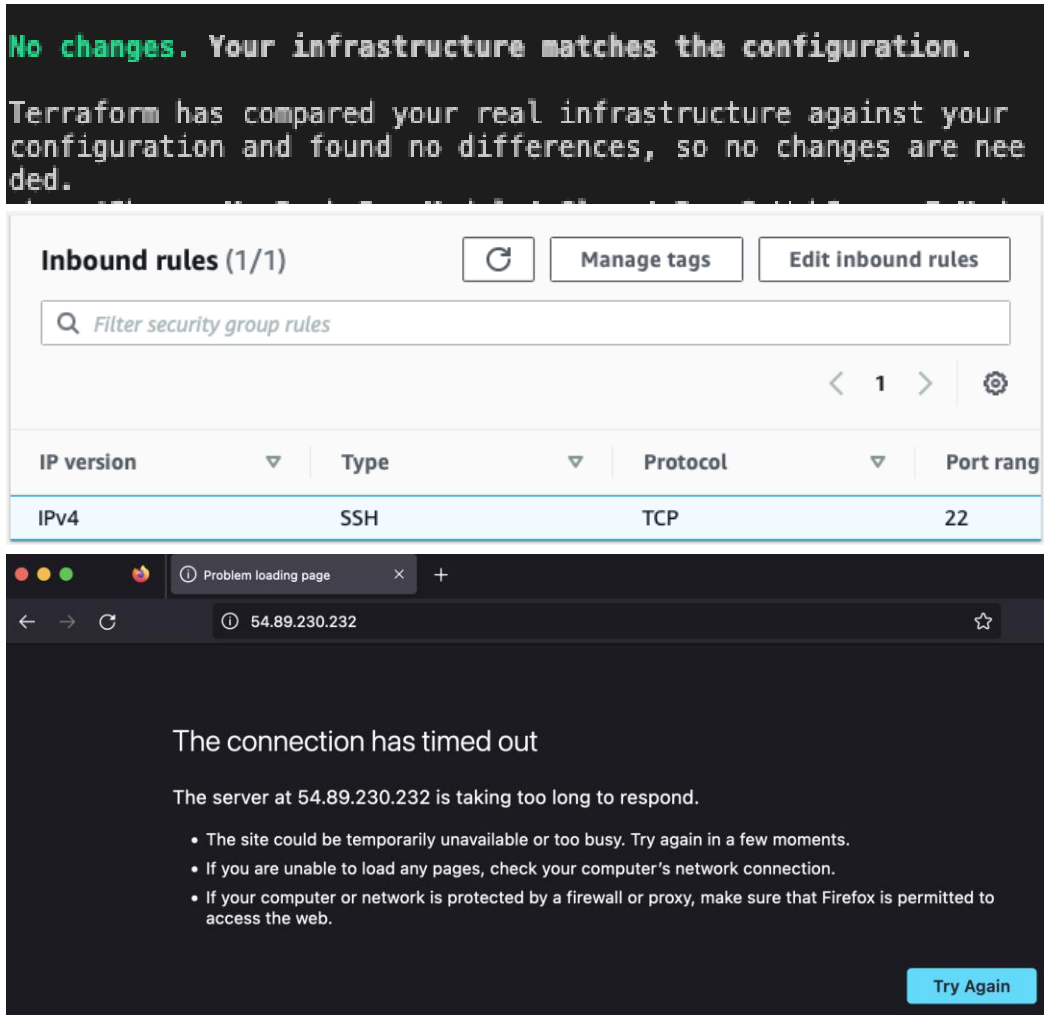
Exercise 4: Drift Detection (1 of 4)

- Continuing with the code from the last exercise, verify that the Apache httpd page is accessible.
- Connect to the AWS Console and find the security group which was created as part of the previous exercise.
- Verify that as per the code in vpc/main.tf, that the inbound rules allow SSH on port 22 and TCP on port 80.



Exercise 4: Drift Detection (2 of 4)

- Go back to the console window where you ran **`terraform apply`** and run **`terraform plan`**
- Review the output, it should show that the deployed infrastructure and the code match.
- Edit the inbound rules in the AWS Console, delete rule TCP on port 80.
- Try to refresh the Apache httpd home page.
- It will take some time, but eventually it will fail to load.



Exercise 4: Drift Detection (3 of 4)

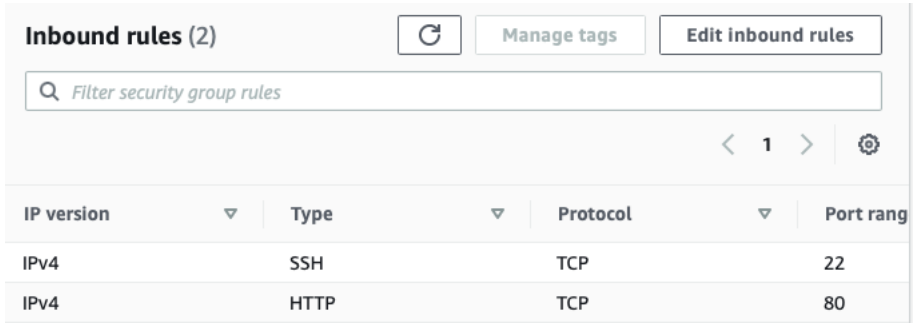
- Go back to the console window where you ran **`terraform apply`** and run **`terraform plan`** again
- This time terraform clearly identifies that there is a difference between the code and the infrastructure, that the Security Group needs a new rule added.
- In this way, terraform can help identify drift detection, where the configuration of the infrastructure has been modified manually and is no longer consistent with the code.

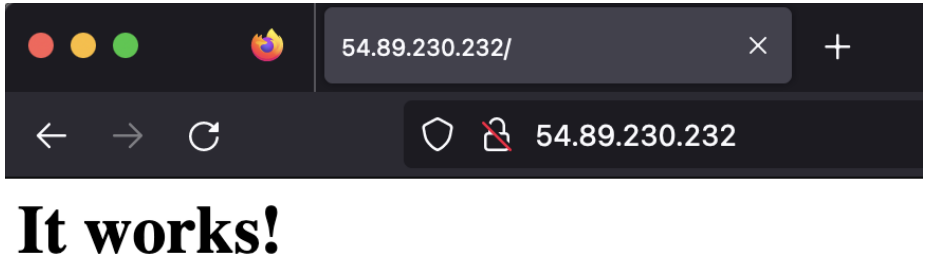
```
Terraform will perform the following actions:

  # module.vpc.aws_security_group.tf_public_sg will be updated in-place
  ~ resource "aws_security_group" "tf_public_sg" {
        id                     = "sg-0c62e2d5f5a5aba34"
      ~ ingress                = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + description      = "allow traffic from TCP/80"
              + from_port        = 80
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "tcp"
              + security_groups  = []
              + self             = false
              + to_port          = 80
            },
            # (1 unchanged element hidden)
        ]
        name                   = "tf_public_sg"
        tags                   = {
            "Name" = "Terraform-SecurityGroup"
        }
        # (7 unchanged attributes hidden)
    }

Plan: 0 to add, 1 to change, 0 to destroy.
```

Exercise 4: Drift Detection (4 of 4)

- Run **`terraform apply`** again
- The code executes quickly and reports that one item has changed.
- Checking the inbound rules through AWS Console and you can see that the rule has been reinstated.
- Finally, check the Apache httpd home page and confirm that it is loading now without problems.
- Terraform can be used to alert admins to changes in configuration and then used to remediate as shown here.

```
Plan: 0 to add, 1 to change, 0 to destroy.
module.vpc.aws_security_group.tf_public_sg: Modifying... [id=sg-0c62e2d5f5a5aba34]
module.vpc.aws_security_group.tf_public_sg: Modifications complete after 1s [id=sg-0c62
e2d5f5a5aba34]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
```





Exercise 5: Automated Testing (2 of 7)

- First step is to install Go. Select your OS from this resource. Run installation verification step.

  https://go.dev/doc/install

- Clone the sample code from GitHub
- Review the terraform code. Note some differences from the last exercise in the user_data section of compute>main.tf. These highlighted lines configure httpd to run as ec2-user and creates the index.html file. The default httpd “It Works!” page returns a HTTP status of 403.

```
<<-EOF
#!/bin/bash
sudo yum update -y
sudo yum install httpd -y
sudo usermod -a -G apache ec2-user
sudo chown -R ec2-user:apache /var/www
sudo chmod 2775 /var/www
sudo find /var/www -type d -exec chmod 2775 {} \;
sudo find /var/www -type f -exec chmod 0664 {} \;
sudo echo "<html><body><h1>Automated Terraform Testing</h1></body></html>" >> /var/www/html/index.html
sudo service httpd start
EOF
```

Exercise 5: Automated Testing (1 of 7)

Overview

- Test automation is a critical element of operating in a Cloud native world. There is lots of content that highlights the importance of different types of test automation for application development, but it is a less common practice for infrastructure as code.
- In this exercise, we will take some existing code and verify (using Terratest) that when our code is executed, we get the expected outcome.

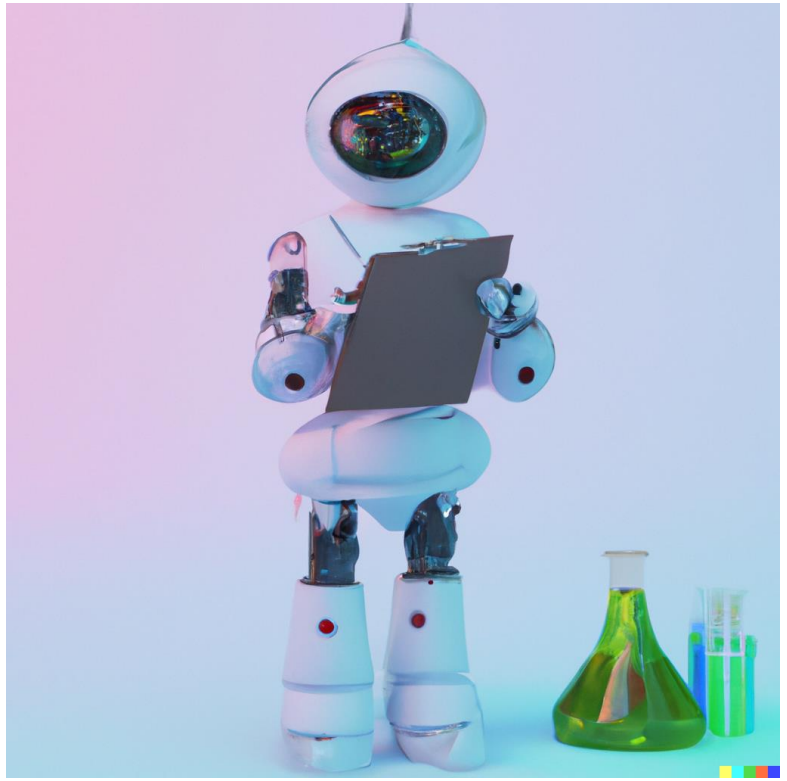
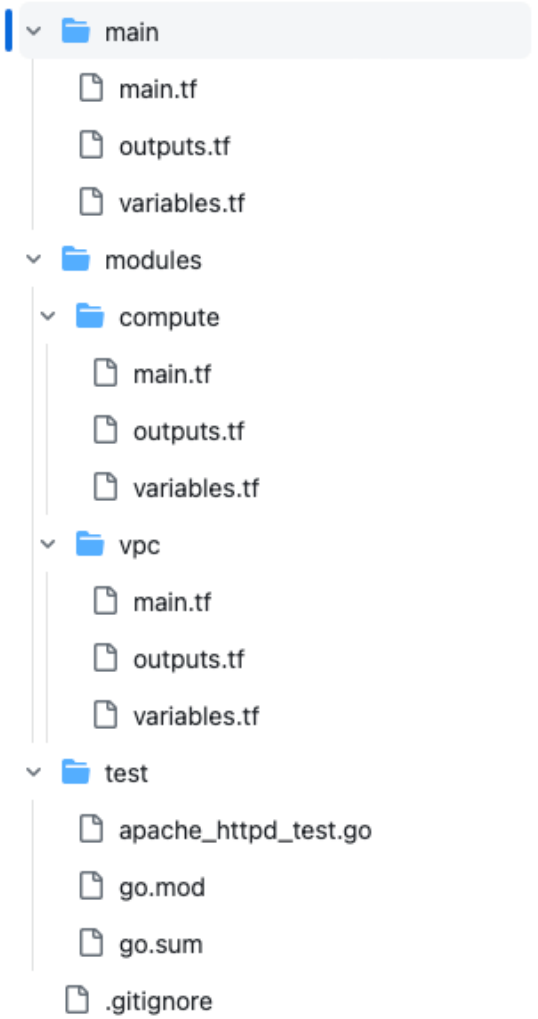


Image: DALL-E

Exercise 5: Automated Testing (3 of 7)

- You will also notice that the top-level files in this modularized codebase have been moved into a main folder. This is to align with Terratest convention. The modules remain as before.
- A test folder has been added which contains the go code to test the infrastructure created by the terraform code.
- The Terratest framework is quite straightforward to use, the linked video provides a good overview. *
- You do not need to know the go to get something working here. You can build your knowledge from this starting point.



*https://www.youtube.com/watch?v=xhHOW0EF5u8 provides a walkthrough of several Terratest scenarios.

Exercise 5: Automated Testing (4 of 7)

This example is simplified to highlight the essential elements to code and execute this test.

- There are no run time inputs required
- The defer command ensures that this code will be executed at the end, even if there is a crash during execution of other code.
- InitAndApply runs the familiar `terraform init` and `terraform apply` steps.
- validateApacheHttpdServer is covered on the next slide

```go
// An example of a unit test for the Terraform to create an EC2 with
// a Apache httpd server
func TestApacheHttpdServer(t *testing.T) {

    t.Parallel()

    terraformOptions := &terraform.Options{
        // The path to where our Terraform code is located
        TerraformDir: "../main",
    }

    // At the end of the test, run `terraform destroy` to clean up any resources
    // that were created
    defer terraform.Destroy(t, terraformOptions)

    // This will run `terraform init` and `terraform apply` and fail the test
    // if there are any errors
    terraform.InitAndApply(t, terraformOptions)

    // Check that the app is working as expected
    validateApacheHttpdServer(t, terraformOptions)
}
```

Exercise 5: Automated Testing (5 of 7)

```go
// Validate the Apache httpd server is working as expected
func validateApacheHttpdServer(t *testing.T, terraformOptions *terraform.Options) {

    // Run `terraform output` to get the url for the webserver
    url := terraform.Output(t, terraformOptions, "Apache-Webserver-Public-URL")

    // Verify the app returns a 200 OK with the text "Automated Terraform Testing"
    expectedStatus := 200
    expectedBody := "<html><body><h1>Automated Terraform Testing</h1></body></html>"
    maxRetries := 10
    timeBetweenRetries := 3 * time.Second
    http_helper.HttpGetWithRetry(t, url, nil, expectedStatus, expectedBody, maxRetries, timeBetweenRetries)
}
```

- validateApacheHttpdServer is where the testing step is performed
- The value of Apache-Webserver-Public-URL is retrieved from the `terraform output` and assigned to `url`

  e.g., Apache-Webserver-Public-URL = http://54.198.138.215
- Expected HTTP status and response body are used to verify that for the output URL, the deployment is as expected. If we used the default ”It works!” page, then we would specify 403 as the `expectedStatus`.
- The test will retry 10 times, every 3 seconds, until a successful result is found, or all retries are used.

Exercise 5: Automated Testing (6 of 7)

- To execute the test, enter the following command

  go test -v -timeout 15m -run TestApacheHttpdServer

```
shane@Shanes-MacBook-Pro test %  go test -v -timeout 15m -run TestApacheHttpdServer
=== RUN   TestApacheHttpdServer
=== PAUSE TestApacheHttpdServer
=== CONT  TestApacheHttpdServer
TestApacheHttpdServer 2023-10-21T21:13:44-04:00 retry.go:91: terraform [init -upgrade=false]
```

- This kicks off `terraform init` followed by `terraform apply`

```
TestApacheHttpdServer 2023-10-21T21:13:45-04:00 retry.go:91: terraform [apply -input=false -auto-approve -lock=false]
```

- As `terraform destroy` is deferred, the `validateApacheHttpdServer` method is executed next, which retrieves the URL for the hosted page and tries to call it.

```
TestApacheHttpdServer 2023-10-21T21:14:39-04:00 logger.go:66: Apache-Webserver-Public-URL = "http://54.147.6.78"
TestApacheHttpdServer 2023-10-21T21:14:39-04:00 retry.go:91: terraform [output -no-color -json Apache-Webserver-Public-URL]
TestApacheHttpdServer 2023-10-21T21:14:39-04:00 logger.go:66: Running command terraform with args [output -no-color -json Apache-Webserver-Public-URL]
TestApacheHttpdServer 2023-10-21T21:14:40-04:00 logger.go:66: "http://54.147.6.78"
TestApacheHttpdServer 2023-10-21T21:14:40-04:00 retry.go:91: HTTP GET to URL http://54.147.6.78
TestApacheHttpdServer 2023-10-21T21:14:40-04:00 http_helper.go:32: Making an HTTP GET call to URL http://54.147.6.78
```

- After several attempts a successful response is returned and the test execution progresses to `terraform destroy`.

Exercise 5: Automated Testing (7 of 7)

```
TestApacheHttpdServer 2023-10-21T21:34:13-04:00 logger.go:66: Destroy complete! Resources: 8 destroyed.
TestApacheHttpdServer 2023-10-21T21:34:13-04:00 logger.go:66:
--- PASS: TestApacheHttpdServer (133.38s)
PASS
ok      github.com/SMannionYorkU/Module4_Class4_Repo3_Terratest 133.653s
```

When `terraform destroy` completes the result of the test is shown.

This executes quite quickly, taking just over 2 minutes to create the infrastructure, test it and destroy it.

Triggering this test whenever new code is committed is a great quality check that your changes have not broken anything.

This level of testing is mandatory for application developers but should also be part of your practice as a CloudOps engineer, as it maintains quality without manual effort.

Exercise 6: DEV vs PROD (1 of 5)

Code Overview

- As a software engineering principle, it is more effective to have one codebase that is deployed to many different environments, rather than a codebase that is rebuilt for each environment.
- Rebuilding code introduces risk of differences. We want the code that was signed off in testing to be the code that is then deployed to production.
- Similarly, with Terraform, it is preferrable to use exactly the same code for all environments, from DEV through to PROD.

Image: DALL-E

Exercise 6: DEV vs PROD (2 of 5)

- One strategy that can be used to achieve this is input variables to trigger environment specific code execution.
- In the code for this exercise, a new variable called environment is added to the top-level code, with a default value of PROD.
- This value is passed to the compute module, where it is used as follows:
  - To specify environment specific key pair names
  - To select an instance_type of t2.micro for PROD and t3.micro for all other environments
  - To specify an environment specific EC2 tag name

Class4 > Module4_Class4_Repo5_DEV-vs-PROD > variables.tf >

```
#-----variables.tf-----
#=======================
variable "region" {
  type    = string
  default = "us-east-1"
}

variable "environment" {
  type    = string
  default = "PROD"
}
```

```
resource "aws_key_pair" "aws-key" {
  key_name   = format("webserver_%s", var.environment)
  public_key = file(var.ssh_key_public)
}

#Create and bootstrap webserver
#================================
resource "aws_instance" "webserver" {
  instance_type               = var.environment == "PROD" ? "t2.micro" : "t3.micro"
  ami                         = data.aws_ssm_parameter.webserver-ami.value
  tags = {
    Name = format("webserver_tf_%s", var.environment)
  }
```

Exercise 6: DEV vs PROD (3 of 5)

- Executing this code without any input variables creates a t2.micro server with a PROD specific name.

```
terraform apply -auto-approve
```

Name	Instance ID	Instance state	Instance type
webserver_tf_PROD	i-0d26d81b590a112d4	Running	t2.micro

- Executing this code specifying DEV as the environment input variable creates a t3.micro server with a DEV specific name.

```
terraform apply -var environment="DEV" -auto-approve
```

Name	Instance ID	Instance state	Instance type
webserver_tf_DEV	i-0e43ddb0780a18d0d	Running	t3.micro

Exercise 6: DEV vs PROD (4 of 5)

- Another strategy that can be used is terraform workspaces. This allows more than one set of infrastructure to be created using the same code base.
- Firstly, run terraform apply providing “TEST” as an input
- Next create a workspace for a “PATCH” deployment and run terraform apply providing “PATCH” as an input

```
shane@Shanes-MacBook-Pro Module4_Class4_Repo5_DEV-vs-PROD % terraform workspace list
* default

shane@Shanes-MacBook-Pro Module4_Class4_Repo5_DEV-vs-PROD % terraform workspace new patch
Created and switched to workspace "patch"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
shane@Shanes-MacBook-Pro Module4_Class4_Repo5_DEV-vs-PROD % terraform workspace list
  default
* patch

shane@Shanes-MacBook-Pro Module4_Class4_Repo5_DEV-vs-PROD % terraform apply -var environment="PATCH" -auto-approve
```

Exercise 6: DEV vs PROD (5 of 5)

- This approach allows multiple concurrent infrastructure deployments from the same machine.
- Reviewing the AWS Console, both TEST and PATCH deployments are evident, including environment specific key pairs.

Name	Instance ID	Instance state	Instance type
webserver_tf_TEST	i-09508fb0b31a53df9	Running	t3.micro
webserver_tf_PATCH	i-0f85b808610b63d47	Running	t3.micro

Name	Type	Created
webserver_TEST	rsa	2023/10/21 22:23 GMT-4
webserver_PATCH	rsa	2023/10/21 22:25 GMT-4

- This approach works for a locally managed infrastructure but does not scale well, so using variables and building on that approach is a better option for more complex implementations

05.AlbWithSubnetWithMultiAzs

1. specify region, which has azs
2. create vpc in this region
3. create subnets, specify vpc and az – min 2, in different azs
4. create security group for alb, specify vpc
5. create alb, specify subnets and security group

Add the following next

- aws_lb_target_group – specify vpc_id
- aws_lb_ listener– specify alb.arn
- aws_lb_listener_rule – specify aws_lb_ listener

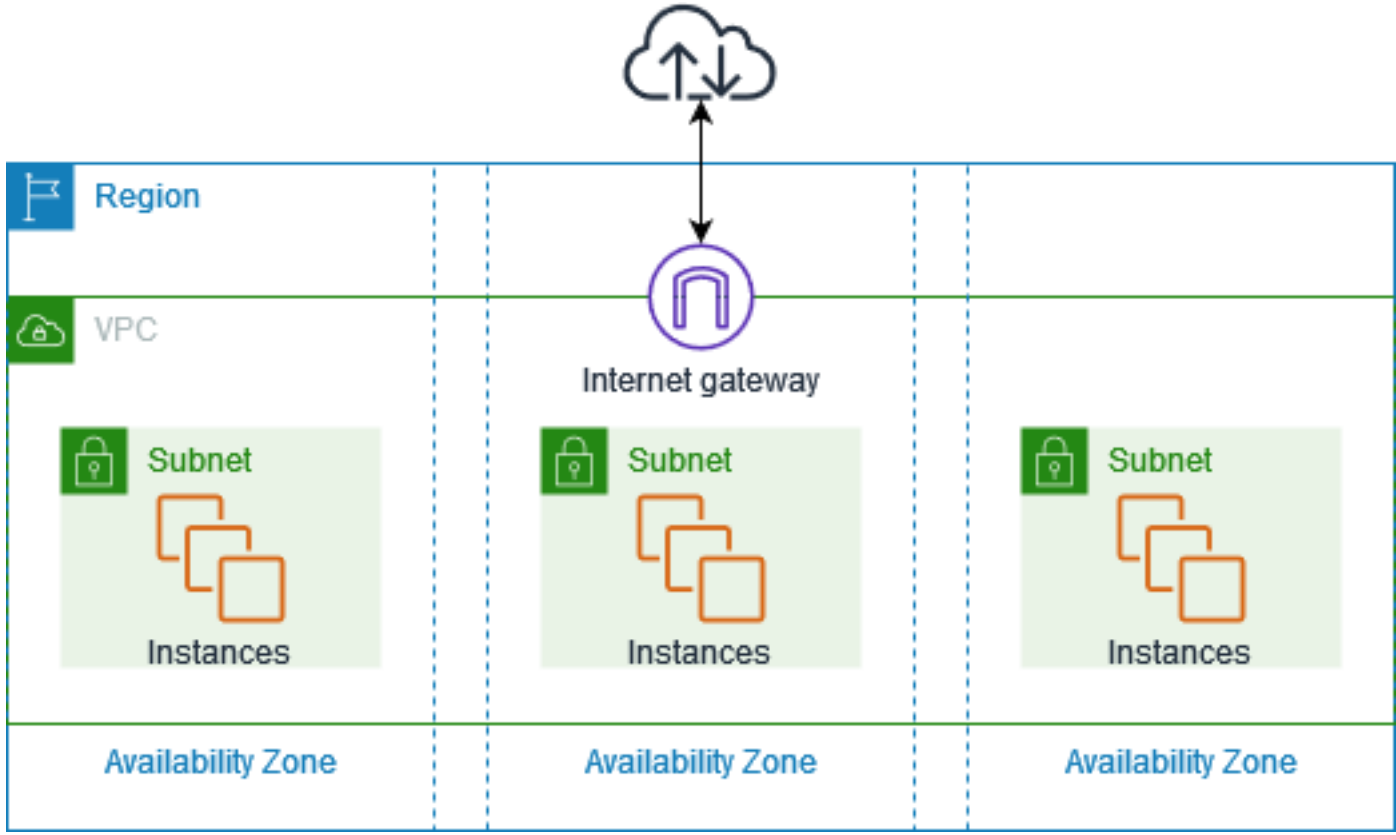
- aws_ssm_parameter
- template_file
- aws_route_table – specify vpc
- aws_route_table_association – change from subnet to gateway here removed this as there was a conflict and may not be needed.

- aws_security_group (not alb)
- aws_launch_configuration
- aws_autoscaling_group

Later

- aws_availability_zones – look at how to improve subnet creation based on azs

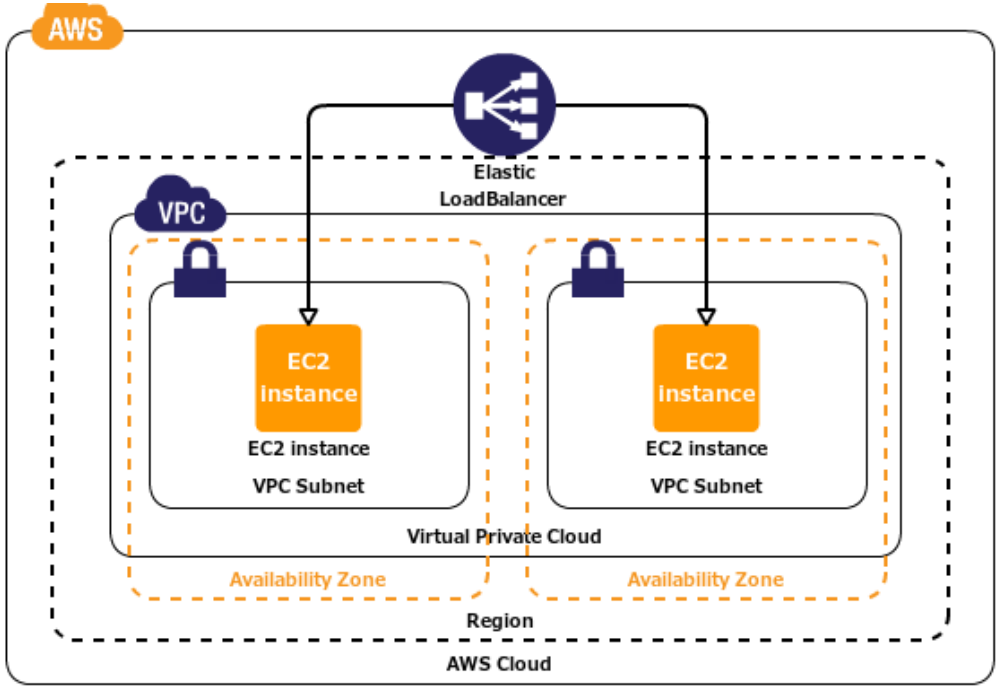
Designing a Multi-Availability Zone AWS Architecture with VPC, Subnets, and ALB (con't)



Exercise 6: Convert Exercise 5 to modules

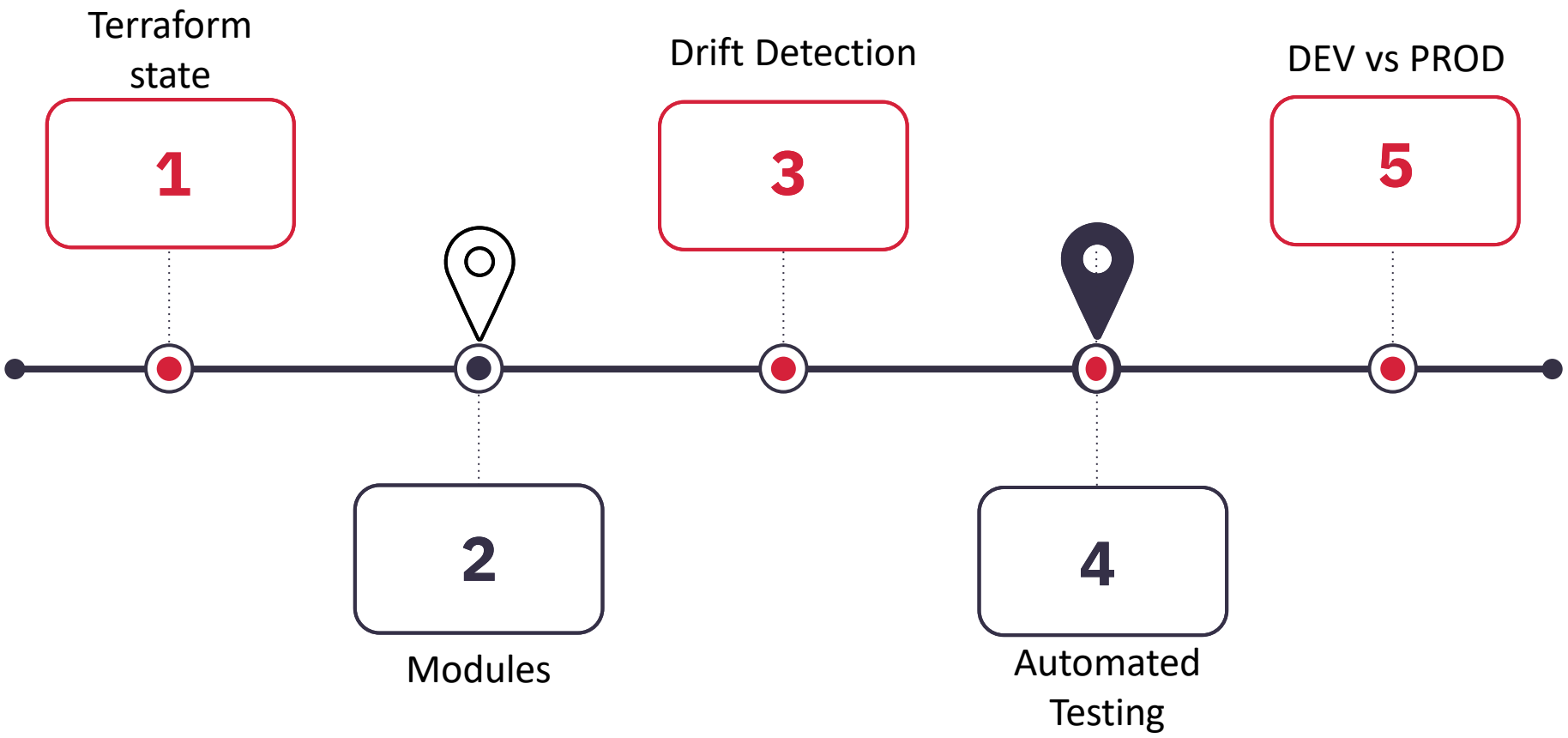
Code Overview

- As the single terraform main.tf file becomes more complex, it is more difficult to read.
- Using modules breaks this large file down into more manageable blocks, which are easier to maintain and to reuse.
- Next is a walkthrough of how the same objective is achieved, but through a decomposed file structure.



https://www.bogotobogo.com/DevOps/Terraform/Terraform-VPC-Subnet-ELB-RouteTable-SecurityGroup-Apache-Server-1.php

Journey Map


Terraform state
1
Drift Detection
3
DEV vs PROD
5
2
Modules
4
Automated Testing
Testing

Wrap up

- Q & A
- What's 1 thing you learned?
- What's 1 challenge?

References

- Images [Slide 1] used under license from Microsoft PowerPoint stock images.
- Image [Slide 18] 2020 "Woman Sitting on Bench Looking at Mountains" by Yan Krukau used under free license from Pexels.